

7_WebAttack_Detection_Using

Step 1: Import Required Libraries

We imported all the libraries needed for reading data, training the model, and plotting graphs.

```
# ML for CBS practicals.
# Make a copy of this notebook and place it under your own
"MyDrive/Colab Notebooks" folder
# Please do not the run code here.

#importing all libraries
import pandas as pd
import numpy as np
from sklearn import preprocessing
import matplotlib.pyplot as plt
%matplotlib inline
```

Step 2: Load Dataset and Display First Row

We loaded the CSV file using pandas and displayed the first 5 rows using `dataset.head()`.

```
#Mounting my google drive here in google colab using the following code
from google.colab import drive
drive.mount('/content/drive')

path_to_dataset = "/content/drive/MyDrive/Colab
Notebooks/Datasets/Intrusion_Dataset/GeneratedLabelledFlows/Thursday-
WorkingHours-Morning-WebAttacks.pcap_ISCX.csv"

dataset = pd.read_csv(path_to_dataset, engine='python',
encoding='latin-1')

dataset.head()
```

	Flow ID	Source IP	Source Port	Destination IP	Destination Port	Protocol	Timestamp	Flow Duration	Total Fwd Packets	Total Backward Packets	...	min_seg_size_forward	Active Mean	Active Std	A
0	192.168.10.3-192.168.10.50-389-33898-6	192.168.10.50	33898.0	192.168.10.3	389.0	6.0	6/7/2017 8:59	113095465.0	48.0	24.0	...	32.0	203985.500	5.758373e+05	162
1	192.168.10.3-192.168.10.50-389-33904-6	192.168.10.50	33904.0	192.168.10.3	389.0	6.0	6/7/2017 8:59	113473706.0	68.0	40.0	...	32.0	178326.875	5.034269e+05	142
2	8.0.6.4-8.6.0.1-0-0-0	8.6.0.1	0.0	8.0.6.4	0.0	0.0	6/7/2017 8:59	119945515.0	150.0	0.0	...	0.0	6909777.333	1.170000e+07	2040
3	192.168.10.14-65.55.44.109-59135-443-6	192.168.10.14	59135.0	65.55.44.109	443.0	6.0	6/7/2017 8:59	60261928.0	9.0	7.0	...	20.0	0.000	0.000000e+00	
4	192.168.10.3-192.168.10.14-53-59555-17	192.168.10.14	59555.0	192.168.10.3	53.0	17.0	6/7/2017 8:59	269.0	2.0	2.0	...	32.0	0.000	0.000000e+00	

5 rows x 85 columns

```
dataset.shape
```

(458968, 85)

Step 3: Data Cleaning and Exploration

We removed extra spaces from column names and dropped the 'Fwd Header Length.1' column.

```
# dropping a few columns
dataset.columns = dataset.columns.str.strip()
dataset = dataset.drop(columns=['Fwd Header Length.1']) #dropping the Fwd Header Length.1
dataset.shape #printing the shape of the dataset
```

(458968, 84)

```
[9] dataset['Label'].unique() #printing the unique attack labels present in the labels
```

array(['BENIGN', 'Web Attack \x96 Brute Force', 'Web Attack \x96 XSS', 'Web Attack \x96 Sql Injection', nan], dtype=object)

```
[10] dataset['Label'].value_counts() #computing the number of samples per attack types
```

Label	count
BENIGN	168186
Web Attack \x96 Brute Force	1507
Web Attack \x96 XSS	652
Web Attack \x96 Sql Injection	21

dtype: int64

```
[11] dataset.columns #printing the various attributes/features present in the dataset
```

Index(['Flow ID', 'Source IP', 'Source Port', 'Destination IP', 'Destination Port', 'Protocol', 'Timestamp', 'Flow Duration', 'Total Fwd Packets', 'Total Backward Packets', 'Total Length of Fwd Packets', 'Total Length of Bwd Packets', 'Fwd Packet Length Max', 'Fwd Packet Length Min', 'Fwd Packet Length Mean', 'Fwd Packet Length Std', 'Bwd Packet Length Max', 'Bwd Packet Length Min', 'Active Mean', 'Active Std', 'Label'], dtype='object', length=25)

Step 4: Handle Missing/Infinity Values

We replaced 'Infinity' with -1 and converted 'Flow Bytes/s' and 'Flow Packets/s' to numeric.

```
# noticed that some attributes have infinite values, therefore
replacing them by -1 using .replace fucntion
dataset.replace('Infinity', -1, inplace=True)

# converting the values present in Flow Bytes/s and Flow Packets into
numeric data
dataset[["Flow Bytes/s", "Flow Packets/s"]] = dataset[["Flow Bytes/s",
"Flow Packets/s"]].apply(pd.to_numeric)

dataset
```

	Flow ID	Source IP	Source Port	Destination IP	Destination Port	Protocol	Timestamp	Flow Duration	Fwd Packets	Backward Packets	...	min_seg_size_forward	Active Mean	Active St
0	192.168.10.3-192.168.10.50-389-33898-6	192.168.10.50	33898.0	192.168.10.3	389.0	6.0	6/7/2017 8:59	113095465.0	48.0	24.0	...	32.0	203985.500	5.758373e+C
1	192.168.10.3-192.168.10.50-389-33904-6	192.168.10.50	33904.0	192.168.10.3	389.0	6.0	6/7/2017 8:59	113473706.0	68.0	40.0	...	32.0	178326.875	5.034269e+C
2	8.0.6.4-8.6.0.1-0-0-0	8.6.0.1	0.0	8.0.6.4	0.0	0.0	6/7/2017 8:59	119945515.0	150.0	0.0	...	0.0	6909777.333	1.170000e+C
3	192.168.10.14-65.55.44.109-59135-443-6	192.168.10.14	59135.0	65.55.44.109	443.0	6.0	6/7/2017 8:59	60261928.0	9.0	7.0	...	20.0	0.000	0.000000e+C
4	192.168.10.3-192.168.10.14-53-59555-17	192.168.10.14	59555.0	192.168.10.3	53.0	17.0	6/7/2017 8:59	269.0	2.0	2.0	...	32.0	0.000	0.000000e+C
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
170361	157.240.18.35-192.168.10.51-443-55641-6	157.240.18.35	443.0	192.168.10.51	55641.0	6.0	6/7/2017 12:59	49.0	1.0	3.0	...	20.0	0.000	0.000000e+C
170362	192.168.10.51-199.16.156.120-45337-443-6	199.16.156.120	443.0	192.168.10.51	45337.0	6.0	6/7/2017 12:59	217.0	2.0	1.0	...	32.0	0.000	0.000000e+C
170363	192.168.10.12-192.168.10.50-60148-22-6	192.168.10.12	60148.0	192.168.10.50	22.0	6.0	6/7/2017 12:59	1387547.0	41.0	46.0	...	32.0	0.000	0.000000e+C
170364	192.168.10.12-192.168.10.50-60146-22-6	192.168.10.12	60146.0	192.168.10.50	22.0	6.0	6/7/2017 12:59	207.0	1.0	1.0	...	32.0	0.000	0.000000e+C
170365	192.168.10.12-192.168.10.50-60146-22-6	192.168.10.50	22.0	192.168.10.12	60146.0	6.0	6/7/2017 12:59	50.0	1.0	2.0	...	32.0	0.000	0.000000e+C

170366 rows x 14 columns

Step 5: Check Class Balance

We checked how many benign vs attack records there are using value_counts().

The dataset was imbalanced: ~168K benign vs 2K attack.

```
[28] benign_total = len(dataset[dataset['Label'] == "BENIGN"]) # computing the
benign_total # print the value
```

168186

```
attack_total = len(dataset[dataset['Label'] != "BENIGN"]) # computing the
attack_total # print the samples
```

2180

+ Код

+ Текст

Calculate undersampling ratio (70% benign, 30% attack)

```
[30] # noticed a highly imbalanced dataset
# Use Enlargement multiplier to extract 70% of benign instances.
enlargement = 1.1 # using enlargement computing the probability of includi
benign_included_max = attack_total / 30 * 70 # need to include
benign_inc_probability = (benign_included_max / benign_total) * enlargemer
print(benign_included_max, benign_inc_probability) # print the number of i
```

5086.666666666667 0.03326872232726466

Build balanced dataset manually

```
import random
indexes = []
benign_included_count = 0
for index, row in dataset.iterrows():
    if (row['Label'] != "BENIGN"):
        indexes.append(index)
    else:
        # Copying with benign_inc_probability
        if random.random() > benign_inc_probability: continue
        # Have we achieved 70% (5087 records)?
        if benign_included_count > benign_included_max: continue
        benign_included_count += 1
        indexes.append(index)
dataset_balanced = dataset.loc[indexes] #creating a balanced dataset
```

+ Код

+ Текст

```
[32] dataset_balanced['Label'].value_counts() #doing a value_counts() to verify
```

count

Label	
BENIGN	5087
Web Attack □ Brute Force	1507
Web Attack □ XSS	652
Web Attack □ Sql Injection	21

Step 6: Under sample and Balance Dataset

We used random under sampling to keep 70% benign and 30% attack data.

Final balanced dataset 5087 benign and 2180 attack records.

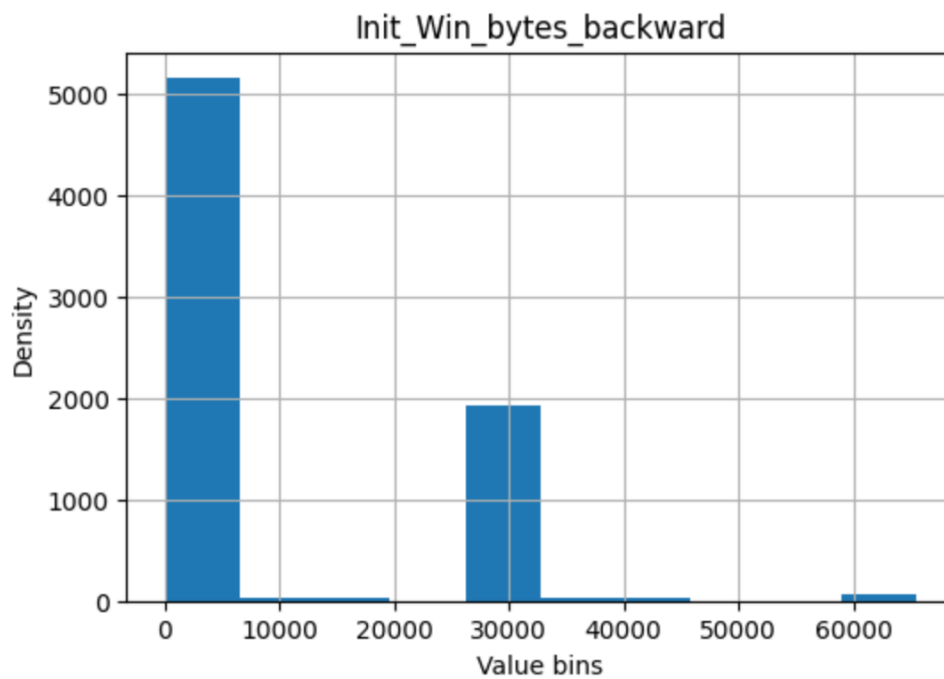
Step 7: Drop Extra columns

We removed unnecessary columns like Flow ID, Ips, Ports, Protocol, Timestamp.

Also dropped **Init_Win_bytes_backward** and forward after checking their histograms.

```
plt.savefig('Init_Win_bytes_backward.png', dpi=300,
```

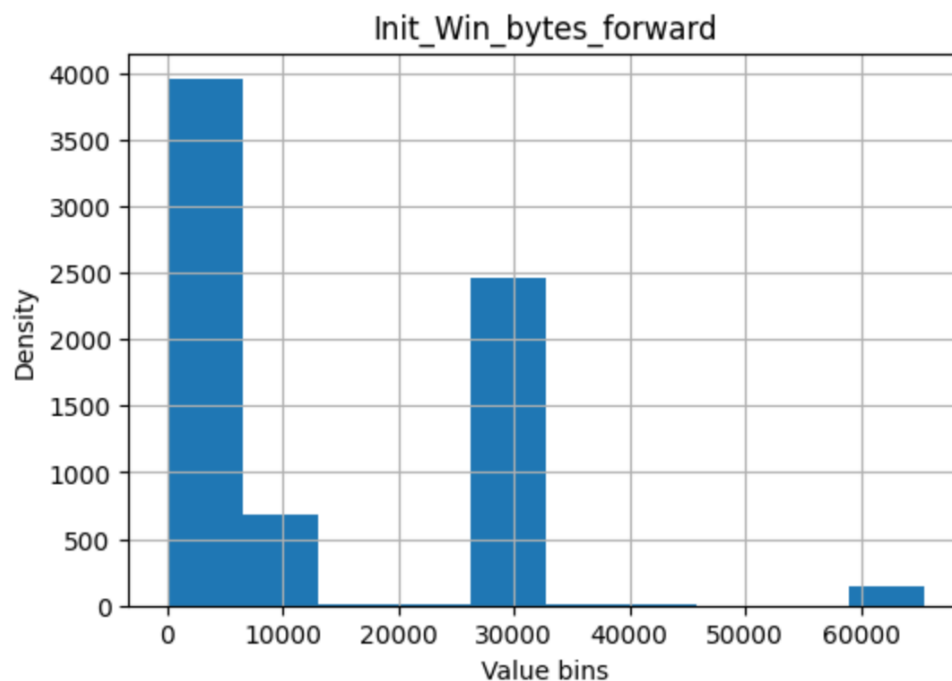
[↕]



▶

```
#visualizing attribute Init_Win_bytes_forward
if 'Init_Win_bytes_forward' in dataset_balanced_attack.columns:
    dataset_balanced_attack['Init_Win_bytes_forward'].hist(figsize=(6,4),
    plt.title("Init_Win_bytes_forward")
    plt.xlabel("Value bins")
    plt.ylabel("Density")
    plt.savefig('Init_Win_bytes_forward.png', dpi=300)
```

[↕]



Step 8: Train/Test Split

We split the dataset into X and Y (features and labels)

Used **train_test_split** with 70% training and 30% testing

Step 9: Train Decision Tree

We created a Decision Tree with **max_leaf_nodes =5**.

Used **cross_val_score** with 10-fold cross validation to test the model.

Step 10: Evaluate the Model

We used **confusion_matrix** and **classification_report** to evaluate the **predictions**

```
⇒ [1 0 1 ... 0 1 0]
   [[1534    4]
    [  93  550]]
```

		precision	recall	f1-score	support
	0	0.94	1.00	0.97	1538
	1	0.99	0.86	0.92	643
	accuracy			0.96	2181
	macro avg	0.97	0.93	0.94	2181
	weighted avg	0.96	0.96	0.95	2181

Step 11: Conclusion

In this practical, we built and tested a Decision Tree model to detect web attacks.

The accuracy was 96% and F1-score for attacks was 0.92.

The model is good, but recall could be improved. We can try Random Forest or more preprocessing to improve results.